Projet 2 - Ticket mystère au supermarché

DAVY + CINDY | INF1007 | PLAN DE TRAVAIL

Plan proposé à Davy et Cindy, fondé sur le PDF et le CSV présents dans ce dossier. Il ne représente pas une validation de l'auxiliaire. Aucun modèle n'a encore été entraîné et aucun score n'est annoncé.

1. Ce que les données permettent

Analyse effectuée sur `supermarket_sales - Sheet1.csv` :

• 1 000 lignes, 17 colonnes, 0 valeur manquante et 0 ligne entièrement dupliquée.

• 1 000 identifiants de facture distincts. Une ligne est une transaction décrite par une catégorie de produits, pas un catalogue d'articles ni le détail d'un panier à plusieurs catégories.

• Période : du 1er janvier au 30 mars 2019 ; heures observées de 10 h à 20 h.

• Trois succursales : A/Yangon (340 lignes), B/Mandalay (332), C/Naypyitaw (328). La ville et la succursale sont redondantes : garder seulement la succursale.

• Six catégories : Fashion accessories (178), Food and beverages (174), Electronic accessories (170), Sports and travel (166), Home and lifestyle (160), Health and beauty (152).

• Clients : Member (501), Normal (499). Paiements : Ewallet (345), Cash (344), Credit card (311).

• Prix unitaire : 10,08 à 99,96 ; quantité : 1 à 10 ; total : 10,6785 à 1 042,65 ; note : 4 à 10.

• Total moyen : 322,966749 ; médiane : 253,848. Ces valeurs décrivent tout le fichier et ne constituent pas une référence d'évaluation : la référence doit être calculée sur l'entraînement uniquement.

• La devise ne peut pas être déduite du CSV : afficher « unités monétaires » tant que la documentation source ne l'a pas confirmée.

Relations vérifiées numériquement :

```
cogs = Unit price × Quantity
Tax 5% = gross income = 0,05 × cogs
Total = 1,05 × Unit price × Quantity
gross margin percentage est constante : environ 4,761905
```

L'écart maximal observé entre le total et sa formule est inférieur à 0,000000000001 (arrondi informatique). Fournir le prix ET la quantité au modèle rendrait la tâche essentiellement calculable. Fournir la taxe, cogs ou gross income révélerait aussi le total.

La note Rating n'est pas une cible prioritaire : ses corrélations linéaires avec les mesures numériques sont très faibles (en valeur absolue, au plus environ 0,037). Cela ne prouve pas l'absence de tout lien, mais ne justifie pas un jeu de satisfaction fiable.

Conformité : taille, format, types et nombre d'entrées conviennent à la proposition ci-dessous. Aucun nom, courriel ou adresse de personne n'apparaît dans les colonnes. Retirer Invoice ID des entrées et ne pas prétendre connaître le caractère réel ou synthétique des données sans source.

Source candidate à confirmer avec votre téléchargement : https://www.kaggle.com/datasets/aungpyaeap/supermarket-sales . La licence n'est pas présente dans le CSV et n'a pas pu être vérifiée sur la page consultée. Noter la licence exacte et ses conditions dans le README avant de redistribuer le CSV dans le dépôt public. Le fichier fait environ 132 ko, donc moins de 25 Mo.

2. Le jeu retenu : Ticket mystère

But : proposer une estimation du montant d'une transaction dont la quantité est cachée, puis comparer son erreur à celle du modèle.

Une partie comporte 5 manches :

1. Tirer sans répétition une transaction dans un ensemble réservé, jamais utilisé pour entraîner le modèle.

2. Afficher ses indices : prix unitaire, catégorie, succursale, type de client, paiement, heure et jour de semaine. Ne pas afficher quantité, taxe, cogs, revenu brut ou total.

3. Le joueur saisit son estimation du total, puis clique sur « Prédire et révéler ».

4. Faire prédire le modèle une seule fois ; révéler total réel, estimation du joueur et prédiction du modèle.

5. Comparer les erreurs absolues : `abs(estimation - total_reel)`. La plus petite gagne un point ; égalité si les erreurs arrondies au centième sont égales. Le point est une règle de jeu, indépendante de la métrique scientifique.

6. Enregistrer la manche, actualiser le graphique et passer à la suivante. Bloquer une deuxième validation de la même manche pour éviter les doublons.

7. Montrer le score final joueur/modèle et permettre une nouvelle partie.

L'incertitude est réelle : le prix ne révèle pas combien d'unités ont été achetées. Le modèle ne doit pas promettre un montant exact. Il s'agit d'une estimation de ticket, pas d'une simulation de stocks, de caisse complète ou de bénéfices futurs.

Ajouter un mode « Simulation libre » pour satisfaire explicitement les saisies demandées : l'utilisateur peut modifier chaque caractéristique et obtenir une prédiction. Ici, pas de total réel connu, donc pas de point ni de comparaison à une vérité inventée. L'historique marque ce mode et laisse les champs de vérité/erreurs vides.

3. Contrat des données à fixer ensemble avant de coder

Utiliser exactement les mêmes noms internes chez les deux membres :

Nom interne	Source	Type et contrôle
prix_unitaire	Unit price	float fini, intervalle des métadonnées
categorie	Product line	catégorie parmi les 6 valeurs
type_client	Customer type	Member ou Normal
succursale	Branch	A, B ou C
paiement	Payment	Ewallet, Cash ou Credit card
heure	Time	entier, 10 à 20 dans le CSV
jour_semaine	Date	catégorie, lundi à dimanche

Cible : `Total`, renommée `total`. Sept entrées : deux numériques et cinq catégorielles. Extraire les dates avec le format explicite `%m/%d/%Y`, et les heures avec `%H:%M`. Ne pas confondre jour/mois. Les libellés français peuvent être séparés des valeurs internes anglaises.

Exclure des entrées : Quantity, Tax 5%, cogs, gross income, gross margin percentage, Invoice ID, Total. Ne pas utiliser Rating ni Gender dans cette première version. City est redondante avec Branch. La date brute est remplacée par le jour de semaine.

Le JSON de métadonnées contient : version du schéma, cible, liste ordonnée des variables, type, libellé, catégories autorisées ou bornes numériques, unité, métrique, score du modèle, score de référence, graine et taille des ensembles. Les catégories et intervalles de validation doivent être définis à partir de l'entraînement ; documenter tout domaine fixé a priori. Vérifier que les manches réservées sont compatibles avec ce domaine, sans recalculer le modèle sur elles.

Le modèle sauvegardé est une Pipeline complète : prétraitement et estimateur. L'interface lui fournit un DataFrame d'une ligne avec les sept noms attendus. Elle ne recrée pas son propre encodage.

Interfaces de programmation proposées :

```
JeuDeDonnees.charger() -> DataFrame
JeuDeDonnees.preparer() -> DataFrame
ModeleTicket.charger(chemin_modele, chemin_metadata) -> None
ModeleTicket.predire(entrees: dict) -> float
Variable.valider(valeur: str) -> float | int | str
Partie.nouvelle_manche() -> dict
Partie.evaluer(estimation_joueur, prediction_modele) -> dict
Historique.ajouter(enregistrement: dict) -> None
Historique.lire() -> list[dict]
```

Les chemins de données et de modèles se construisent avec `os.path.join` depuis la racine du projet, pas depuis le dossier courant du terminal.

4. Répartition équilibrée

L'égalité repose sur la difficulté, les tests, la documentation et l'intégration ; pas sur le nombre de fichiers. Budget initial : 6 h individuelles chacun + 2 h ensemble, donc 8 h d'engagement par personne. C'est une estimation de planification ; la durée de 8 h du sujet est indicative et n'est pas une garantie. Réajuster au point d'intégration si une moitié prend davantage de temps.

Bloc	Davy : données et moteur prédictif	Cindy : jeu et application	Charge par personne
Conception commune	Vérifier colonnes, cible et fuites ; convenir du contrat	Définir règles, écrans et champs ; convenir du contrat	30 min ensemble
Réalisation principale	Données, exploration, prétraitement, séparation, référence, modèle, sauvegarde et métadonnées	Variables et validation, logique de partie, interface, graphique et historique	4 h chacun
Vérification et documentation	6 tests données/modèle, CI, dépendances, sections données/résultats du README	6 tests validation/jeu/historique, sections installation/jeu du README et maquette	2 h chacun
Intégration et revue croisée	Brancher le modèle ; relire la logique du jeu et expliquer ses classes	Brancher l'application ; relire le modèle et expliquer l'évaluation	1 h ensemble
Oral et finalisation	Environ 2 min 30 de parole, revue finale	Environ 2 min 30 de parole, revue finale	30 min ensemble

Davy livre : `donnees.py`, `modele.py`, `entrainer.py`, `test_donnees.py`, `test_modele.py`, workflow CI et fichiers du modèle. Cindy livre : `variables.py`, `exceptions.py`, `jeu.py`, `historique.py`, `interface.py`, `main.py`, tests correspondants. Les deux contribuent réellement à README, DECISIONS et JOURNAL_IA au fil de leurs tâches. Cindy ne porte pas seule les documents ; Davy ne porte pas seul tous les tests.

Chacun relit au moins une fonction de l'autre et écrit ou corrige un test de cette fonction. Alterner pilote/copilote pendant les séances communes. Les commits sont faits sous les vrais comptes respectifs ; ne pas fabriquer des contributions pour donner l'apparence d'un partage.

5. Directives de Davy

1. Lire le CSV avec pandas via un fichier ouvert avec `with`. Vérifier existence, colonnes attendues, types, valeurs finies et doublons. Conserver le CSV brut ; ne pas supprimer des lignes uniquement pour améliorer le score. Expliquer que l'absence de valeurs manquantes a été vérifiée.

2. Contrôler les relations comptables et les plages ; transformer dates/heures ; sélectionner explicitement les sept entrées. Créer une classe JeuDeDonnees dont les méthodes modifient son état protégé, avec constructeur et `__str__`.

3. Faire un histogramme du total, avec titre, axe horizontal « Montant (unités monétaires) » et axe vertical « Nombre de transactions ». L'enregistrer avec `savefig` pour l'oral.

4. Présenter cible, modèle et métrique à l'auxiliaire AVANT l'entraînement final (validation C).

5. Faire `train_test_split(test_size=0.2, random_state=42)` : environ 800 lignes d'entraînement, 200 réservées. Garder la quantité et les colonnes révélatrices hors de X. Ne pas mélanger les lignes réservées à l'entraînement. Sauvegarder les identifiants de lignes réservées pour les manches.

6. Prétraiter les catégories par OneHotEncoder dans un ColumnTransformer, inclus dans une Pipeline avec LinearRegression. Ajuster la Pipeline uniquement sur l'entraînement. Une régression linéaire suffit ; ne comparer un deuxième modèle que si le reste est terminé.

7. Entraîner une référence `DummyRegressor(strategy='mean')` sur les mêmes données d'entraînement. Calculer la RMSE des deux sur les mêmes lignes de test. La RMSE s'exprime dans l'unité du total et pénalise les grosses erreurs.

8. Implémenter aussi `sqrt(mean((y_reel - y_predit)**2))` avec NumPy sans boucle. Tester sa concordance avec la métrique scikit-learn. Ne pas annoncer de résultat avant exécution.

9. Analyser l'écart avec la référence, les erreurs et l'incertitude liée à la quantité cachée. Si le modèle ne bat pas la référence, le dire. Ne pas modifier les entrées ou réglages à répétition en regardant le test ; utiliser une validation interne à l'entraînement si nécessaire.

10. Sauvegarder Pipeline avec joblib et métadonnées JSON. Un lancement normal charge ces fichiers. Prévoir une erreur compréhensible s'ils manquent ; ne pas lancer automatiquement l'entraînement.

11. Employer un décorateur qui mesure le temps d'entraînement et expliquer son utilité dans le README.

12. Préparer requirements.txt et GitHub Actions : installer les dépendances et lancer pytest à chaque push. Pas d'ouverture de fenêtre Tkinter dans les tests CI.

La RMSE n'est pas un pourcentage de confiance. Une régression linéaire ne fournit pas automatiquement une probabilité ou un intervalle fiable : ne pas en inventer. Les limites d'extrapolation ou les prédictions négatives éventuelles doivent être documentées ; si une correction de sortie est décidée, évaluer exactement la même règle sur le test et dans l'application.

6. Directives de Cindy

1. Proposer Tkinter + ttk : fenêtre locale, champs simples, intégration Matplotlib. Dessiner une maquette papier et faire approuver la technologie et la maquette avant de développer l'interface (validation D).

2. Créer une classe abstraite Variable avec `valider`, puis VariableNumerique et VariableCategorielle qui redéfinissent cette méthode. Constructeurs, état protégé, annotations et docstrings. La variable numérique

précise si un entier est requis.

3. Créer DonneeInvalideError. Refuser vide, texte non numérique, NaN, infini, valeurs hors bornes, décimales dans l'heure et catégories inconnues. Afficher une erreur en français, sans traceback.

4. Créer les widgets à partir du JSON, pas d'une deuxième liste codée en dur. Les champs des manches sont préremplis puis verrouillés ; les mêmes champs sont éditables en simulation libre. L'estimation du joueur est un champ distinct, non transmis au modèle.

5. Créer Partie : liste des manches disponibles, manche active, état révélé/non révélé, nombre de manches et scores protégés. Séparer cette logique de Tkinter pour pouvoir la tester.

6. Créer Application qui contient un ModeleTicket, des objets Variable, une Partie et un Historique : c'est la composition demandée. Elle appelle la logique ; elle ne contient ni entraînement ni formule de métrique.

7. Après révélation, intégrer un graphique Matplotlib à trois barres : joueur, modèle, réel ; titre « Comparaison des montants », axe vertical « Montant (unités monétaires) », catégories explicites. En simulation, afficher seulement la prédiction, sans réel inventé. Les widgets et le graphique se mettent à jour sans multiplier les fenêtres.

8. Écrire l'historique CSV avec `with`, en UTF-8, en-tête une seule fois. Colonnes proposées : horodatage, mode, sept entrées, estimation joueur, prédiction, réel, erreurs, gagnant, numéro de manche. Pour une simulation libre, les champs sans vérité sont vides.

9. Ajouter un onglet historique consultable. Un fichier absent correspond à un historique vide. Une écriture impossible affiche un message clair sans annoncer que la sauvegarde a réussi. Ne pas faire disparaître la prédiction déjà calculée.

10. Écrire `main.py` pour lancer l'application avec garde `if __name__ == '__main__'`. Importer un module ne doit jamais ouvrir de fenêtre.

Maquette de référence : en haut titre et onglets Jeu / Simulation / Historique ; à gauche champs des indices ; au centre estimation joueur, bouton et résultat ; à droite graphique ; en bas score et bouton Manche suivante. Afficher « Exercice pédagogique : ces estimations ne constituent pas une aide à la décision financière » pour lever l'ambiguïté du thème monétaire.

7. Tests à se partager (12 proposés, minimum du sujet : 8)

Responsable	Cas à tester
Davy 1	Chargement d'un petit CSV valide et extraction correcte des colonnes
Davy 2	Fichier absent et colonne obligatoire absente : exceptions attendues
Davy 3	Date 1/5/2019 interprétée comme 5 janvier ; heure extraite correctement
Davy 4	Aucune colonne révélatrice dans X ; identifiants entraînement/test disjoints
Davy 5	RMSE NumPy égale à celle de scikit-learn avec pytest.approx sur des valeurs connues
Davy 6	Prédiction identique avant/après sauvegarde et chargement, avec pytest.approx
Cindy 1	Valeur numérique normale et bornes exactes acceptées
Cindy 2	Vide, texte, hors bornes, NaN et infini rejetés avec pytest.raises
Cindy 3	Catégorie autorisée acceptée, catégorie inconnue refusée
Cindy 4	Gagnant correct dans les cas joueur, modèle et égalité, exemples calculés à la main
Cindy 5	Une manche validée ne peut pas être comptée deux fois ; fin à cinq manches
Cindy 6	Historique absent puis deux ajouts et relecture ; pas d'en-tête dupliqué, simulation sans réel

Utiliser tmp_path pour les fichiers de test, de petits exemples indépendants et des attentes calculées à la main. Les tests peuvent être paramétrés. La mesure de couverture n'exige aucun seuil ; publier le résultat réellement obtenu.

8. Structure et commandes visées

```
README.md
DECISIONS.md
JOURNAL_IA.md
requirements.txt
.gitignore
.github/workflows/tests.yml
donnees/supermarket_sales.csv
modeles/ticket.joblib
modeles/metadonnees.json
modeles/manches_test.csv
graphiques/exploration_total.png
src/__init__.py
src/donnees.py
src/modele.py
src/entrainer.py
src/variables.py
src/exceptions.py
src/jeu.py
src/historique.py
src/interface.py
src/main.py
tests/...
```

Les fichiers sont à créer pendant le développement ; cette liste n'affirme pas qu'ils existent. Le CSV brut actuel reste la référence. Ignorer dans Git .venv, __pycache__, .pytest_cache, .coverage et l'historique personnel des parties. Ne pas ignorer les fichiers nécessaires pour lancer la démo.

Commandes à rendre fonctionnelles depuis la racine du futur dépôt :

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m src.entrainer
.\.venv\Scripts\python.exe -m src.main
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m coverage run --source=src -m pytest
.\.venv\Scripts\python.exe -m coverage report -m
```

Dépendances à prévoir : pandas, numpy, matplotlib, scikit-learn, joblib, pytest, coverage. Tkinter est fourni par de nombreuses distributions Python ; vérifier sa disponibilité avec `python -m tkinter`, ce n'est pas un paquet à ajouter arbitrairement à pip. Tester puis consigner les versions utilisées.

9. Étapes communes et validations

1. A et B : présenter le thème du supermarché, le fichier, ses 1 000 lignes, ses types et sa licence. Comparer brièvement avec deux pistes écartées. Expliquer pourquoi ce jeu utilise des transactions plutôt qu'un catalogue inventé.

2. C : présenter la cible Total, la quantité cachée, les sept entrées, la régression linéaire, la RMSE et la référence moyenne. Faire confirmer que ce problème répond aux attentes avant l'entraînement final.

3. D : présenter Tkinter, la maquette et les deux modes avant développement de l'interface.

4. Commencer le dépôt GitHub public partagé, inviter l'autre membre et obtenir un premier test vert. Ajouter les tests et commits au fil des tâches.

5. Premier point de raccordement : Davy fournit un exemple d'entrées, un modèle chargeable et un JSON ; Cindy vérifie immédiatement la construction des champs et l'appel predire. Fixer le contrat avant d'aller plus loin.

6. Revue croisée et rééquilibrage : si le moteur est terminé mais l'interface encore lourde, Davy prend un test ou l'historique avec accord sur le fichier ; si l'interface est prête, Cindy aide aux métadonnées et à leur validation. Garder des responsabilités claires pour éviter les écrasements.

7. Tester une installation dans un dossier neuf avec les seules instructions du README. Vérifier absence de chemins absolus personnels et fonctionnement sans réentraînement au lancement. Vérifier la dernière CI verte.

Chaque fonction a une docstring précisant entrées, sortie et échecs ; annotations de type ; fonctions de calcul avec return. Utiliser des try/except précis et lever l'exception personnalisée avec raise. Vérifier l'existence avant lecture et utiliser with pour les ouvertures. Garder au moins trois classes avec constructeur, méthodes agissant sur leur état et encapsulation pertinente, l'héritage, le polymorphisme, la composition et une méthode spéciale.

10. Documents et oral

README : but du jeu et règles, provenance et licence, limites des données, installation et commandes, structure et classes, entrées/cible et exclusions, split et graine, RMSE modèle et référence, couverture réelle, décorateur choisi, au moins trois pages officielles effectivement consultées. Liens de départ :

• pandas : https://pandas.pydata.org/docs/user_guide/10min.html

• scikit-learn, Pipeline et ColumnTransformer : https://scikit-learn.org/stable/modules/compose.html

• Tkinter : https://docs.python.org/3/library/tkinter.html

• Matplotlib dans Tkinter : https://matplotlib.org/stable/gallery/user_interfaces/embedding_in_tk_sgskip.html

• pytest : https://docs.pytest.org/en/stable/

DECISIONS.md (1 à 2 pages) : colonnes Choix / Options envisagées / Choix retenu et justification / Validation par l'auxiliaire et date. Ne pas remplir une date ou un accord fictif.

JOURNAL_IA.md : au moins cinq entrées honnêtes avec Outil / Demande / Réponse / Vérification / Décision. Une entrée possible dès maintenant : proposition de prédire Total avec prix et quantité, vérification de la formule sur le CSV, modification pour cacher la quantité et retirer les colonnes comptables. Décrire seulement les vérifications réellement effectuées ; ajouter les autres entrées pendant le travail, sans en inventer cinq après coup.

Oral : 5 minutes d'exposé puis 5 minutes de questions ; maximum quatre diapositives facultatives ; démo obligatoire. Proposition équilibrée : Davy 0:00-1:45 (données et modèle), Cindy 1:45-3:15 (démo : manche, saisie invalide et graphique), Davy 3:15-4:00 (conception et tests), Cindy 4:00-4:45 (historique et limites), puis 15 secondes chacun pour le bilan. Chacun doit expliquer tout le code. Préparer des captures ou une courte vidéo de secours et lancer l'application avant de commencer.

Remise : lien du dépôt public sur Moodle, documents complétés et dernier commit vert. Les dates de remise et de présentation sont vides dans l'énoncé : les vérifier sur Moodle.

11. Liste de contrôle finale

□ Source et licence exactes vérifiées ; redistribution autorisée ou téléchargement documenté.

□ Validations A/B/C/D discutées et tracées honnêtement.

□ 7 entrées, numériques et catégorielles, champs issus du JSON.

□ Quantité et variables comptables révélatrices exclues du modèle et des indices.

□ Split reproductible ; transformations ajustées sur entraînement ; manches hors entraînement.

□ Référence triviale, RMSE sur test et limites expliquées.

□ Pipeline joblib et métadonnées JSON, aucun entraînement au lancement.

□ Jeu 5 manches, simulation libre, saisies robustes et absence de double comptage.

□ Graphique exploratoire sauvegardé et graphique Matplotlib intégré.

□ Historique enregistré et consultable, erreurs de fichier gérées.

□ POO, modules, types, docstrings, os, with, exceptions et décorateur présents.

□ Au moins 8 tests pertinents ; couverture mesurée ; CI verte.

□ Commits réguliers des deux membres et compréhension mutuelle.

□ README complet, DECISIONS 1-2 pages, journal IA 5 entrées dont une décision modifiée après vérification.

□ Installation vérifiée dans un dossier neuf et démo de secours prête.